

Data Platform as a Service

System purpose:

Our system helps developers store, analyze, modify, and process through several built-in layers that enable rapid, flexible custom experiments to discover data potential and shared storage scaling with security features.

Achievement summary:

- Users can select the optimal configurations to preprocess data.
- Works well with other tools
- Users can clean data and filter useful structures.
- Compatible, stable, and reliable when storing and retrieving from the database
- Organize massive unstructured data, turn these insights into actionable information to improve business decision-making.

The domain that our system operates in is data services clients, we prioritize serving:

- Small to medium size company
- Developers
- System analysts
- DB administrators
- Employees
- Data scientists
- Warehouse staff

System scope:

What does the system do and does not do?

What our system does:

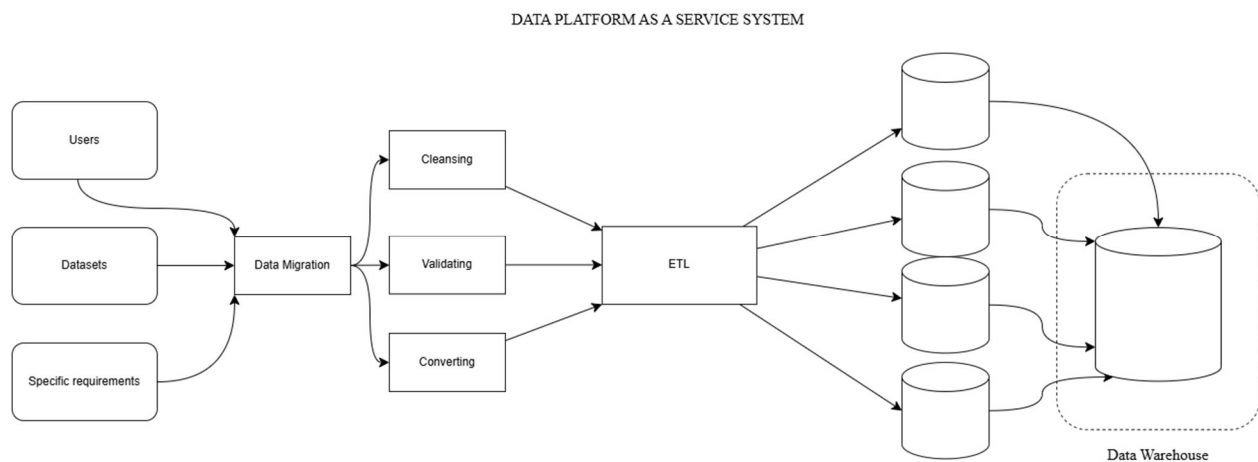
- Store and track user records
- Connect to schemas, test connections, and table visualization
- Generate basic reports
- Choose tables and scheduling
- Recognize user SQL queries
- Filter and save as a dataset
- Dataset sharing, REST API direct call, can download results

What our system does not do (out of scope):

- Profit optimization
- Password manager
- Analyze sales

System functional:

- Create new dataset record (or file upload)
- Remove dataset
- View system loggings
- Running new experiment
- Editing experiment
- Undo experiment
- View experiment results
- Generate experiment reports



SLDC context:

Agile builds on iterative refinement while waterfall gives small improvements. When we merge these two ideas together, we have a system that optimizes both speed and clear structured logic of fast build inherit all the best possible performance. To confirm this approach, we might write a module to see how it performs when both are combined.

Gantt chart

Project Time Line

Task	March			April			May		
System planning	5 days								
User requirements clarification		3 days							
Design data migration			7 days						
Test plan				10 days					
Fix bugs					4 days				
Scheduling & user approve						3 days			
Evaluation							5 days		
System documentation								3 days	

Risk assessment matrix

R1	User access information they should not see			
R2	User may lost data with accidental delete			
R3	User forgot to save data			
R4	User log in but forgets where their model stored			
R5	User confuse about experiment results			

Resource plan

	Number of people	How long	Which tasks
System analyst	1	All	
Software engineer	3	All phases	Coding, connect to databases
Quality assurance	1	Design and implement phase	Check system quality
Product manager	1	All phases	Checks product aligns with end users want
UI/ UX designer	1	Design and implement phases	Verify and validates the interface design easy to use
Solution architect	1	All	Verify API

Hardware:

16 - 32 GB RAM

600 GB – 2 TB SSD

5 vCPUs

Software

Kafka

Storage Server

4 vCPUs

25 TB SSD

Extra cloud storage

Amazon S3

Project cost estimate: \$4800 - \$9,600

Schedule: Entire team members should focus on implementing and work from the hard part first. This means everyone has responsibility to get things moving by setting up a meeting on a weekly basis.

Installation strategy method:

Chosen Method: parallel

We have decided to implement this method because it can override the older version and the system still works correctly

Feasibility:

Can we achieve building the system?

- With finance support
- With the resource plan
- With working prototype
- With entire team understanding the problem we worked on
- With the right architectural setup

Data migration and conversion

ETL process

Where can we get the data?

So for our system to accurately measure it needs to read from spreadsheets, reads from user file,

Data cleansing

We begin to filter the data

Removes repeat patterns

Checks where the data has shown error in formatting

Validation

Ensure the data meets checkpoint and save as a file so we can reuse again later

System testing

Test plan

Test scope:

What do we want to test?

- New experiment

Checks user if they want to select new experiment

Check users have logged in to the system and user has enough space

Let user continue if they have met the training specifications, otherwise they will be unable to run new experiment since the environment does not have enough memory

- Dataset CRUD

Allows user to manipulate what to do with their own dataset

For our users, they have the ability to

Upload or either add dataset

View dataset

Changes dataset

Delete the dataset from the system

- Billing generation

Tracks users' memory use, and processing

Applies costs if extra time and additional cleansing

Use flag to track the payment states

Calculate billing and let them know

- Generate experiment reports

Connect the users with experiment table and result table

Debug and error handling strategy

Identify and prove that the modules with inputs can lead to same behavior as the expected output

The use of automated testing tools will help the system validate and verify the test cases faster and the information reuse is much more efficient

Two data quality issues are missing, and the type of data has appeared many times within the system

Two strategies for data cleansing are reimplementing fix where it makes it incomplete, and removes unwanted repeat data

Testing and validation:

Must test cases by cases, smallest values to largest, tricky edge,

Integration & Evaluation

- Summarize how all components work together to ensure successful system implementation.

Key success factors:

1. The program must able to opens up and run successfully when clone git
2. User can describe, use and process the dataset
3. The user must able to access the system after logging and training

Post implementation review criteria

1. Fix bugs
2. checks database consistency
3. maintain low memory

4. users can clean their dataset

5. users can integrate datasets and save it to a database.

Lesson learned:

Scalable system and making the system viable is hard since it requires time and practical tradeoffs.

What can we update to improve in the future?

We can improve the architecture by using different unit tests, cover more diverse test cases and learn how change architecture can help with system performance, and avoid recreating the same issues.